# OmniCreator AI Studio

## Production AI Agent Build Specification

Version 3.0 - full-fledged production application handoff

| Field | Production instruction |
|---|---|
| Document purpose | This is the direct build file for a coding AI agent. It removes founder pitch material and describes the complete production application: pages, flows, services, database, APIs, editors, AI integrations, publishing, billing, security, deployment, and QA. |
| Prepared date | May 8, 2026 |
| Application type | Multi-tenant SaaS web application for AI video creation, AI image/post creation, professional editing, brand kits, asset management, social publishing, analytics, billing, admin operations, and safety. |
| Non-negotiable rule | No fake product functionality. No demo-only generation. No placeholder buttons. No "coming soon" core features. No simulated media outputs in the shipped product. Every user-visible feature listed here must connect to real application logic and real production services. |
| AI integration rule | The app must use real provider integrations through an AI Gateway. If credentials or provider approvals are missing, the app must show a setup-required state and must not pretend that generation or publishing succeeded. |
| Agent execution rule | The agent must build the complete application described in this file and continue implementation until every production completion requirement passes. |

### COPY THIS COMMAND TO THE AI CODING AGENT

Build OmniCreator AI Studio exactly according to this Production AI Agent Build Specification. Treat this file as the source of truth. Build the complete production SaaS application, not a prototype and not a pitch deck. Use the specified stack, repository structure, pages, flows, services, database tables, API contracts, editor engines, AI Gateway, real provider adapters, render pipeline, publishing pipeline, billing, security, admin tools, QA, and deployment. Do not ship placeholder screens, fake outputs, simulated AI generation, or incomplete core flows. If an external provider credential or platform approval is missing, implement the real adapter and show a setup-required state until the credential is supplied. Continue until all production completion requirements pass.

# Table of Contents


- 1. Direct Verdict on the Earlier Blueprint
- 2. Product Definition in One Page
- 3. Agent Execution Rules
- 4. Production Scope and Feature Coverage
- 5. Complete User Roles and Permissions
- 6. All Pages, Routes, and Screen Requirements
- 7. End-to-End Product Flows
- 8. Video Editor Production Specification
- 9. Image Editor Production Specification
- 10. AI Gateway and Real Provider Integration
- 11. Prompt System and Agentic Workflows
- 12. System Architecture and Repository Structure
- 13. Database Schema
- 14. Backend APIs and DTOs
- 15. Frontend Architecture
- 16. Media, Render, Export, and Storage Pipeline
- 17. Social Publishing and Scheduling
- 18. Billing, Plans, Credits, and Usage Ledger
- 19. Analytics and Recommendation Loop
- 20. Security, Privacy, Safety, Consent, and Rights
- 21. Admin, Support, Observability, and Operations
- 22. Production QA and Test Requirements
- 23. Deployment and DevOps
- 24. Complete Build Backlog for the AI Agent
- 25. Production Completion Requirements
- Appendix A. Environment Variables
- Appendix B. Provider Adapter Checklist
- Appendix C. Editor Keyboard Shortcuts
- Appendix D. Error States
- Appendix E. Official Documentation Links to Verify During Build

# 1. Direct Verdict on the Earlier Blueprint

The earlier blueprint is valuable as a founder/product planning document, but it is not strict enough for a coding AI agent to build the full application without interpretation. It explains the idea, market, roadmap, editor direction, AI provider strategy, and high-level modules, but a production build needs exact pages, services, schemas, operations, acceptance requirements, deployment rules, and real provider integration contracts.

This version is rewritten as an implementation command file. The business pitch is removed. The application defaults are chosen inside the file so the agent does not pause for basic product decisions. The agent should only require external secrets, platform approvals, payment accounts, cloud credentials, and brand/name assets that cannot be invented inside code.

| Question | Answer for this production file |
|---|---|
| Is the earlier file enough for an AI agent? | No. It can start planning, but it is too strategic. This version is the correct handoff format for an AI coding agent. |
| Does the video editor need more detail? | Yes. This version defines tracks, timeline state, clip operations, captions, audio, effects, color, AI tools, rendering, export rules, and tests. |
| Does the image editor need more detail? | Yes. This version defines canvas state, pages, artboards, layers, masks, selections, typography, generative edits, templates, exports, and tests. |
| Does the app work just by buying AI APIs? | No. APIs are only providers. The app still needs orchestration, queues, storage, rendering, billing, editor state, safety, publishing, analytics, and ops. |
| Should the shipped app contain simulated outputs? | No. User-facing product flows must use real application logic and configured providers. Missing credentials must produce setup-required states, not fake success. |
| Should the agent build only a small MVP? | No. The agent must build the full production application described here. Internal implementation order can be staged, but the delivered application must include the complete scope. |

## 1.1 What this file deliberately removes

- Long founder narrative, market essay, investor pitch, and naming brainstorm sections.
- Ambiguous MVP-only language that could cause the agent to stop after a partial product.
- Provider suggestions without implementation contracts.
- Editor descriptions without operation schemas.
- Generic roadmap language without build tasks.
- Any instruction that allows fake generation, fake publishing, or demo-only product behavior.

## 1.2 What this file adds

- Strict production execution command for the AI agent.
- Complete route map and screen-by-screen UI requirements.
- Complete video editor and image editor functional coverage.
- Chosen technical stack and repo structure.
- Database tables, relationships, indexes, and enum values.
- API endpoints with request and response DTOs.
- AI Gateway adapter interface for real providers.
- Asynchronous job architecture with retries, idempotency, billing, safety, and observability.
- Publishing pipeline with OAuth, preflight validation, scheduled jobs, status polling, and manual fallback where APIs are not approved.
- QA, deployment, security, privacy, admin, and production completion requirements.

# 2. Product Definition in One Page

OmniCreator AI Studio is a production SaaS platform that lets creators, agencies, small businesses, and teams generate, edit, publish, and analyze video and image content from one workspace. The product combines AI strategy, scripts, storyboards, image generation, video generation, voiceover, captions, lip-sync/avatar options, manual video editing, manual image editing, publishing, scheduling, analytics, billing, and brand memory.

| Product pillar | Required production behavior |
|---|---|
| Idea to publish | A user can enter a brief, generate a plan, generate assets, edit, export, schedule/publish, and view analytics without leaving the app. |
| Video creation | Short-form and marketing video projects with AI-generated scenes, voice, captions, timeline editing, render/export, and platform variants. |
| Image/post creation | Social posts, thumbnails, ads, carousels, product mockups, brand graphics, image edits, background work, exports, and platform variants. |
| Professional control | AI provides drafts; human users can manually adjust scripts, scenes, timelines, layers, captions, text, effects, audio, colors, masks, exports, and publishing metadata. |
| Brand memory | Brand kits, tone, colors, fonts, logos, product references, character references, forbidden terms, and style examples influence generation and editing. |
| Real provider orchestration | External AI providers are hidden behind one AI Gateway with routing, retries, cost tracking, fallback, moderation, and usage ledger. |
| Publishing and analytics | OAuth accounts, preflight validation, scheduling, direct posting where allowed, status tracking, performance metrics, and AI recommendations. |
| Multi-tenant SaaS | Organizations, workspaces, projects, roles, billing, seats, usage, admin console, audit logs, and security controls. |

## 2.1 Default product decisions

| Decision | Chosen default |
|---|---|
| Product codename | OmniCreator AI Studio. |
| Primary customer | Small agencies, serious creators, ecommerce/social teams, educators/coaches, and startup marketing teams. |
| Platform focus | Social-first video and image content for TikTok, YouTube Shorts, Instagram Reels/posts, LinkedIn posts, Facebook posts, Pinterest pins, and exports for unsupported platforms. |
| Default editor target | CapCut/Canva ease plus professional controls, with enough depth for production social and marketing content. |
| Core AI categories | LLM, image generation/editing, video generation/editing, text-to-speech, transcription, music/SFX, lip-sync/avatar, moderation, translation, embeddings. |
| Default stack | TypeScript monorepo with Next.js web app, NestJS API, Prisma/PostgreSQL, Redis/BullMQ workers, S3-compatible storage, FFmpeg render workers, Stripe, and real AI/social provider adapters. |

# 3. Agent Execution Rules

**The agent must treat these rules as mandatory.**

- Build the complete production application described in this file. Internal implementation order may be incremental, but the delivered product must include all required modules and flows.
- Do not ask the founder for decisions that are already specified here. Use the defaults in this file.
- Do not create fake user-visible functionality. A button must either perform its real action or show a clear setup-required/error state with the reason.
- Do not generate pretend media. All generation must go through a real configured provider adapter or return setup-required.
- Do not mark a feature finished if it only works with local hardcoded data. Use the database, services, queues, storage, and permission system.
- Do not place secrets in code. Use environment variables and the secret manager.
- Do not ignore cost. Every AI, render, publish, and storage-heavy job must be tracked in the usage ledger.
- Do not publish duplicate social posts on retry. Every publishing job must use idempotency keys and status reconciliation.
- Do not allow unsafe voice, likeness, copyright, impersonation, or harmful content workflows. Use consent and safety gates.
- Do not delete user media silently. Use soft delete, retention windows, audit logs, and recoverability where appropriate.

## 3.1 External things the agent cannot invent

| External dependency | Required behavior in the app |
|---|---|
| AI provider API keys | Build env variables, credential validation, provider health checks, setup UI, and adapters. Without keys, show setup-required. |
| Social app approvals | Build OAuth flow, scopes, callback handling, app review documentation, and manual export fallback. Without approval, direct publishing is disabled with a clear reason. |
| Stripe account | Build Stripe integration and test/live env separation. Without account keys, |

| | |
|---|---|
| | billing setup shows required configuration. |
| Cloud infrastructure credentials | Build Terraform/docker/env setup. Without credentials, local dev can run core app but production deploy requires configured cloud keys. |
| Trademark/domain/final brand | Use OmniCreator as codename. Allow brand name/logo/theme to be configured later. |

## 3.2 Production-grade means

- Authenticated multi-tenant app with organizations, workspaces, roles, and billing boundaries.
- Persistent database records for projects, assets, prompts, jobs, renders, publishing, analytics, usage, and audit logs.
- Real job queues for AI generation, media processing, rendering, publishing, analytics ingestion, and scheduled tasks.
- Object storage for uploads, generated assets, thumbnails, proxies, exports, and attachments.
- Autosave and version history for video timelines and image canvases.
- Error handling, retries, rate-limit handling, timeouts, idempotency, and user-visible recovery states.
- Secure token storage and encrypted provider/social tokens.
- Automated tests, render tests, E2E flows, security checks, observability, and deploy/rollback process.

# 4. Production Scope and Feature Coverage

| Module | Required production coverage |
|---|---|
| Authentication | Email/password, magic link or OAuth login, session management, password reset, email verification, organization creation, invite flow, role assignment, account deletion request. |
| Organizations/workspaces | Multi-tenant orgs, client workspaces, members, roles, billing owner, project ownership, workspace-level brand kit and defaults. |
| Projects | Video, image, carousel, campaign, template, and asset-only project types; duplicate, archive, restore, search, tags, status. |
| Create wizard | Guided brief collection for video, image, carousel, ad, thumbnail, product mockup, repurpose, and campaign package. |
| AI plan | Brief analysis, audience, hooks, script, storyboard, layout plan, prompt pack, estimated credits, safety warnings, editable before generation. |
| Generation studio | Runs real provider jobs, shows job status, variants, previews, cost, provider, retry/fallback status, select/save outputs. |
| Video editor | Full social-first multi-track editor with clips, captions, audio, transitions, effects, AI tools, export variants, autosave, undo/redo. |
| Image editor | Layer/canvas editor with pages, artboards, text, shapes, images, masks, selections, AI edits, templates, smart resize, exports. |
| Asset library | Uploads, generated assets, folders, tags, search, favorites, rights metadata, source prompts, provider/model, version history, deletion/restore. |
| Brand kits | Logos, colors, fonts, tone, audience, prompts, product refs, character refs, voice refs, forbidden terms, style examples, brand safety. |
| Templates | Video templates, post templates, thumbnail templates, carousel templates, brand-locked templates, template duplication, category filters. |
| Publishing | OAuth social accounts, preflight validation, scheduling, direct posting where allowed, manual export fallback, calendar, status, analytics links. |
| Analytics | Post metrics, creative metrics, generation metrics, editor usage, retention indicators, recommendations, workspace reporting. |
| Billing | Plans, credits, seats, invoices, usage ledger, spending caps, overage handling, refund/no-charge failed jobs, admin controls. |
| Safety/compliance | Moderation, consent, rights metadata, provenance, watermarks/disclosures where needed, reporting/takedown flow, audit logs. |
| Admin/ops | Provider status, queue dashboard, job retry tools, user/org support tools, cost dashboard, feature flags, incident banner, logs/traces. |

## 4.1 Required content types

| Content type | Required features |
|---|---|
| Short video | 9:16, 1:1, 16:9; 15s, 30s, 60s, custom; generated scenes; voice; captions; timeline edit; MP4 export; publish/schedule. |
| Image post | Instagram/Facebook/LinkedIn/social sizes; generated image; text/layout; layer edit; PNG/JPG/WebP export; publish/schedule. |
| YouTube thumbnail | High-contrast layout, face/product/background support, hook text, A/B variants, export 1280x720 and custom. |
| Carousel | Multi-page canvas, script-to-slides, consistent layout, page rearrange, export pages and PDF, publish if platform supports. |
| Ad creative | Audience angle, brand-safe copy, multiple variants, product scene generation, CTA, platform exports, performance tags. |
| Product mockup | Upload product, remove background, generate realistic setting, shadows/reflections, brand CTA, variants. |
| Campaign package | Generate video, thumbnail, post, carousel, captions, hashtags, schedule, analytics tags in one project. |

# 5. Complete User Roles and Permissions

| Role | Permissions |
|---|---|
| Owner | All organization permissions, billing, deletion, provider settings, security, members, audit logs, data export. |
| Admin | Manage workspaces, members, brand kits, templates, publishing accounts, providers except billing ownership unless granted. |
| Editor | Create/edit projects, generate assets, edit timelines/canvases, export, submit for approval, view analytics. |
| Publisher | Approve and publish/schedule content, manage social accounts if granted, view publishing status. |
| Viewer/client | View projects and preview links, comment/approve where invited, cannot generate or alter billing. |
| Support admin | Internal app operator with scoped support access, impersonation requires audit log and owner-visible record. |

## 5.1 Permission rules

- Every request must resolve user, organization, workspace, role, and resource ownership before access.
- All asset URLs must be signed and time-limited unless intentionally public for social publishing ingestion.
- Publishing accounts belong to a workspace or organization and cannot be used by unauthorized users.

- Billing and provider credentials are visible only to owner/admin roles with explicit permission.
- Client review links use scoped tokens with expiration, optional password, watermark, and download permissions.

# 6. All Pages, Routes, and Screen Requirements

The application must contain these production pages. No listed page should be a blank placeholder. Every primary action must be wired to real backend logic, even when the action returns setup-required because credentials are absent.

| Route | Screen | Required contents |
|---|---|---|
| / | Public landing page | Hero, product demo video area, feature sections, pricing CTA, login/signup, compliance footer. |
| /pricing | Pricing | Plan comparison, credit explanation, FAQ, checkout CTA, enterprise contact. |
| /auth/signup | Signup | Account creation, email verification, organization setup redirect. |
| /auth/login | Login | Email/password or OAuth, forgot password, session recovery. |
| /onboarding | Onboarding | Create organization, choose user type, connect first brand kit, optional provider/social setup. |
| /app | Dashboard | Recent projects, create button, assets, scheduled posts, usage credits, provider health, quick templates. |
| /app/projects | Projects | Search, filters, sort, project cards/table, duplicate, archive, restore. |
| /app/projects/new | Create wizard | Choose video, image, carousel, ad, thumbnail, product mockup, campaign package, repurpose. |
| /app/projects/[id]/brief | Brief editor | Audience, platform, tone, goal, style, duration, language, brand kit, references. |
| /app/projects/[id]/plan | AI plan | Hooks, script, storyboard, layout, prompt pack, cost estimate, editable sections, approve. |
| /app/projects/[id]/generate | Generation studio | Real jobs, variants, provider, status, retry, fallback, select/save, safety warnings. |
| /app/projects/[id]/video-editor | Video editor | Preview, tracks, tools, properties, captions, assets, AI assistant, export. |
| /app/projects/[id]/image-editor | Image editor | Canvas, pages/artboards, layers, tools, properties, AI edits, resize, export. |
| /app/projects/[id]/publish | Publish/export | Platform variants, metadata, captions, hashtags, schedule, preflight, approval, export. |
| /app/projects/[id]/analytics | Project analytics | Post status, metrics, recommendations, variant comparison, export report. |
| /app/assets | Asset library | Folders, search, upload, tags, rights, generated history, previews, bulk actions. |
| /app/brand-kits | Brand kits | List, create, edit logos/colors/fonts/tone/product refs/characters/style samples. |
| /app/templates | Template library | Video/image/carousel/ad templates, categories, preview, duplicate, brand-lock. |
| /app/calendar | Publishing calendar | Calendar/list view, drafts, scheduled posts, failed jobs, reschedule, approve. |
| /app/social-accounts | Social connections | OAuth connections, scopes, token status, disconnect, workspace ownership. |
| /app/analytics | Workspace analytics | Performance by platform, campaign, format, hook, template, posting time. |
| /app/billing | Billing and usage | Plan, payment method, invoices, credits, usage, spending caps, upgrade/downgrade. |
| /app/settings | Workspace/org settings | Members, roles, security, notifications, API keys, data export, deletion request. |
| /app/providers | AI provider settings | Credentials, models, routing priorities, health, limits, enterprise approvals. |
| /app/admin | Internal admin console | Users/orgs, provider health, queue status, job search, cost dashboards, incidents. |

## 6.1 Universal UI requirements

- Responsive desktop-first design with usable tablet layouts. Mobile supports review, approvals, scheduling, and light edits; full editors are desktop-optimized.
- Dark mode and light mode with accessible contrast.
- Global command menu for projects, assets, actions, settings, and help.
- Global notifications for completed generations, failed jobs, publish results, low credits, provider issues, and approvals.
- Credits display in top bar with estimated cost before expensive actions.
- Every destructive action requires confirmation and audit logging.
- All long operations show progress, queue status, and recoverable failure states.

# 7. End-to-End Product Flows

## 7.1 Video creation flow

- User creates a video project and chooses platform, duration, language, goal, tone, audience, brand kit, references, and style.
- AI plan generates hooks, script, CTA, scene list, visual prompts, audio plan, caption style, thumbnail concepts, and estimated credits.
- User edits and approves the plan. The app creates generation jobs for text, image/video scenes, voiceover, music/SFX, captions, and optional avatar/lip-sync.
- Generation studio displays real provider status, variants, previews, costs, warnings, and final selections.
- Timeline is created automatically with selected clips, voiceover, captions, music, transitions, and placeholders only for items that are still actively processing.

- User edits timeline manually: trims, splits, reorders, captions, text, overlays, audio, effects, color, motion, and platform safe zones.
- User exports platform variants or opens publish screen. Preflight validates duration, aspect, codec, file size, captions, thumbnail, metadata, and platform account readiness.
- User publishes/schedules where API allows or exports/manual fallback where API is unavailable. Analytics are collected after publishing.

## 7.2 Image/post flow

- User creates image/post, thumbnail, carousel, ad, or product mockup project.
- AI plan generates layout concept, headline, visual hierarchy, prompt pack, copy, CTA, style, and platform sizes.
- Generation jobs create real images/backgrounds/product scenes/characters/variations through provider adapters.
- Image editor opens editable pages/artboards with layers, text, shapes, images, masks, AI edit operations, templates, and brand controls.
- Smart resize creates platform variants with layout adjustment and safe zones.
- User exports PNG/JPG/WebP/PDF as appropriate or publishes/schedules through connected accounts.

## 7.3 Campaign package flow

- User enters campaign goal, product/service, audience, offer, channels, dates, budget notes, and brand kit.
- AI creates a campaign package: 1-3 short videos, thumbnails, 3-10 image posts, carousel, captions, hashtags, posting schedule, and analytics tags.
- Each asset is created as linked child projects with shared campaign metadata and shared brand context.
- Publishing calendar schedules assets and analytics groups results by campaign.

## 7.4 Approval flow

- Editor submits project for approval with comments and version snapshot.
- Approver/client receives secure review link or in-app notification.
- Approver can comment by timeline timecode, canvas page/layer, or whole project.
- Approved content moves to publish/export. Rejected content returns to editor with required changes.

# 8. Video Editor Production Specification

The video editor must be a real editor, not only a preview screen. It must support manual editing and AI-assisted editing. The first production release should be social-first, but the architecture must support advanced editing without rewriting the project model.

## 8.1 Video editor layout

| Area | Required UI |
|---|---|
| Top bar | Project name, autosave status, undo/redo, comments, credits, share/review, export/publish, settings. |
| Left panel | Assets, templates, brand kit, AI tools, stock/generated media, uploads, audio, captions. |
| Center preview | Playback, safe zones, aspect preview, overlays, selected object controls, frame stepping. |
| Bottom timeline | Tracks, clips, captions, audio waveforms, thumbnails, markers, keyframes, zoom controls. |
| Right properties | Selected clip/layer/caption/audio/effect properties, transform, color, audio, AI actions. |
| AI assistant panel | Structured edit commands: shorten, add captions, remove silence, generate B-roll, reframe, improve hook. |

## 8.2 Timeline features

| Feature area | Required production behavior |
|---|---|
| Tracks | Video, audio, voice, music, SFX, captions, text, image, overlay, adjustment, and marker tracks. |
| Clips | Add, move, trim, split, duplicate, delete, lock, mute, hide, group, ungroup, replace media, show source. |
| Editing modes | Select, blade/split, trim, ripple delete, snap, drag, slip/slide where feasible, keyboard nudging. |
| Preview | Proxy-based playback, waveform and thumbnails, frame stepping, playback speed, safe zones, aspect switch. |
| Captions | Auto captions, manual edit, word-level timing, style presets, highlight words, karaoke captions, translation. |
| Audio | Volume, fade in/out, ducking under voice, normalize loudness, mute/solo, waveform, noise reduction provider action. |
| Transitions | Cut, fade, dissolve, slide, wipe, zoom, whip, glitch, custom duration, preview before apply. |
| Effects | Transform, crop, blur, sharpen, vignette, LUT, brightness, contrast, saturation, speed, reverse, freeze frame. |
| Motion | Position/scale/rotation/opacity keyframes, easing, basic motion presets, text animation presets. |
| Color | Basic correction, LUTs, curves/wheels later, shot match, brand color overlay presets. |
| AI tools | Rough cut, hook doctor, auto B-roll, silence remover, caption styling, auto reframe, scene consistency check, variants. |
| Autosave | Timeline JSON saved regularly and on every meaningful operation with version history. |
| Undo/redo | Command stack for all editor actions, including timeline and property changes. |
| Export | MP4 H.264, 1080p, platform ratios, bitrate presets, thumbnails, captions burned-in or sidecar where supported. |

## 8.3 Timeline document schema

```
type TimelineDocument = {
  id: string;
  projectId: string;
  version: number;
  fps: 24 | 25 | 30 | 60;
  width: number;
  height: number;
  durationMs: number;
  aspectRatio: "9:16" | "1:1" | "16:9" | "4:5" | "custom";
  tracks: TimelineTrack[];
  markers: TimelineMarker[];
  globalStyle: { brandKitId?: string; captionStyleId?: string; safeZones: SafeZone[] };
  renderSettings: RenderSettings;
  createdAt: string;
  updatedAt: string;
};

type TimelineTrack = {
  id: string;
```

```
  type: "video" | "audio" | "voice" | "music" | "sfx" | "caption" | "text" | "overlay" | "adjustment";
  name: string;
  order: number;
  locked: boolean;
  muted: boolean;
  hidden: boolean;
  clips: TimelineClip[];
};

type TimelineClip = {
  id: string;
  assetId?: string;
  kind: "media" | "caption" | "text" | "shape" | "effect" | "generated-placeholder";
  startMs: number;
  durationMs: number;
  sourceStartMs?: number;
  sourceEndMs?: number;
  transform: { x: number; y: number; scale: number; rotation: number; opacity: number };
  crop?: { x: number; y: number; width: number; height: number };
  audio?: { volume: number; fadeInMs: number; fadeOutMs: number; ducking?: boolean };
  caption?: { text: string; words?: CaptionWord[]; styleId: string };
  effects: EffectInstance[];
  transitions: { in?: Transition; out?: Transition };
  keyframes: Keyframe[];
  metadata: Record<string, unknown>;
};
```

## 8.4 Video editor operations

| Operation | Required behavior |
|---|---|
| timeline.addAsset | Insert asset into selected track or create track. Generate proxy/thumb/waveform if missing. |
| timeline.trimClip | Adjust clip edge with snap and minimum duration validation. |
| timeline.splitClip | Split selected clip at playhead and preserve effects/keyframes. |
| timeline.moveClip | Move clip to track/time with collision rules and snapping. |
| timeline.deleteRange | Delete selected clips or time range; optional ripple close gap. |
| timeline.applyCaptionStyle | Apply caption style to selected captions or entire caption track. |
| timeline.generateCaptions | Create transcription job, align word timestamps, add editable caption clips. |
| timeline.autoReframe | Create reframed variant using subject tracking/provider/local CV where available. |
| timeline.createVariant | Duplicate timeline and apply hook/pacing/caption/CTA changes without destroying original. |
| timeline.export | Create render job with idempotency key, expected cost, export settings, and progress updates. |

## 8.5 Video editor AI tools

| AI tool | Input | Output |
|---|---|---|
| AI Rough Cut | Script, storyboard, generated assets, selected duration. | Full timeline draft with scenes, voice, captions, music, transitions. |
| Hook Doctor | First 3-5 seconds of timeline, script, target platform. | Alternative intro text/visual pacing/thumbnails and optional edited timeline variant. |
| Auto B-roll | Script lines and timeline gaps. | Generated or recommended clips inserted as B-roll on overlay track. |
| Silence Remover | Voice/audio track. | Detected pauses and proposed ripple cuts with review before apply. |
| Auto Captions | Audio or script. | Caption clips with word timing and style. |
| Auto Reframe | Timeline and target aspect. | New timeline variant with subject-centered crop and safe zones. |
| Scene Consistency Checker | Generated scene assets and character/product refs. | Drift warnings and regeneration suggestions. |
| One-click Variants | Approved timeline. | 3 or more duplicated variants with different hooks, captions, pacing, CTA, thumbnails. |

## 8.6 Video editor acceptance requirements

- A user can create a 15-60 second video from a brief, generate real assets, open the timeline, edit clips/captions/audio, export MP4, and publish or schedule to a connected supported platform.
- Timeline state survives refresh, logout/login, and browser crash through autosave/versioning.
- Undo/redo works for clip movement, trim, split, caption edits, text changes, effects, and property changes.
- Render job produces playable MP4 with correct aspect, duration, audio sync, captions, and no black frames unless intentional.

- Failed provider/render jobs show actionable error states and never charge credits twice.

# 9. Image Editor Production Specification

The image editor must be a real layer/canvas editor for posts, thumbnails, ads, carousels, and product mockups. It must combine Canva-style speed, Photoshop-style layer control for social content, and AI-native editing.

## 9.1 Image editor layout

| Area | Required UI |
|---|---|
| Top bar | Project name, autosave, undo/redo, comments, share/review, resize, export/publish. |
| Left panel | Assets, templates, uploads, brand kit, AI tools, text, shapes, elements, stock/generated media. |
| Center canvas | Selected page/artboard, guides, grid, safe zones, rulers optional, zoom/pan. |
| Bottom/page strip | Pages for carousel/multi-size variants with duplicate/reorder/delete. |
| Right panel | Layer properties, transform, typography, effects, masks, color, export settings. |
| Layers panel | Layer stack, groups, lock/hide, rename, blend mode, opacity, clipping, mask status. |

## 9.2 Image editor features

| Feature area | Required production behavior |
|---|---|
| Canvas/pages | Single and multi-page projects, artboards, templates, custom dimensions, grids, safe zones, zoom/pan. |
| Layers | Text, image, shape, vector/basic path, background, adjustment, group, smart object/reference asset. |
| Layer operations | Add, move, resize, rotate, crop, duplicate, delete, group, align, distribute, lock, hide, reorder, opacity. |
| Text | Brand fonts, typography controls, line height, letter spacing, alignment, shadows, outlines, fills, text effects. |
| Selections/masks | Background remove, subject selection, rectangular/ellipse/freehand selections, masks, feather, invert. |
| AI editing | Generate image, generative fill, inpaint, outpaint, object remove, background replace, product scene, style match. |
| Retouching | Healing, clone/basic cleanup, denoise/upscale provider action, color match, brightness/contrast/saturation. |
| Templates | Post, story, carousel, thumbnail, ad, product mockup templates with brand-lock options. |
| Smart resize | Generate platform sizes and preserve hierarchy using layout constraints and brand-safe zones. |
| Exports | PNG, JPG, WebP, transparent PNG, PDF/carousel export, asset pack ZIP where useful. |
| Autosave/versioning | Canvas JSON saved regularly with revision history and recoverable versions. |

## 9.3 Canvas document schema

```
type CanvasDocument = {
  id: string;
  projectId: string;
  version: number;
  pages: CanvasPage[];
  sharedAssets: string[];
  brandKitId?: string;
  createdAt: string;
  updatedAt: string;
};

type CanvasPage = {
  id: string;
  name: string;
  width: number;
  height: number;
  background: Fill;
  safeZones: SafeZone[];
  layers: CanvasLayer[];
};

type CanvasLayer = {
  id: string;
  type: "text" | "image" | "shape" | "group" | "adjustment" | "mask" | "smart-object";
  name: string;
```

```
  visible: boolean;
  locked: boolean;
  opacity: number;
  blendMode: "normal" | "multiply" | "screen" | "overlay" | "soft-light" | "difference";
  transform: { x: number; y: number; width: number; height: number; rotation: number };
  style: LayerStyle;
  mask?: LayerMask;
  children?: CanvasLayer[];
  text?: TextPayload;
  image?: { assetId: string; crop?: Rect; sourcePromptId?: string };
  shape?: ShapePayload;
  metadata: Record<string, unknown>;
};
```

## 9.4 Image editor operations

| Operation | Required behavior |
|---|---|
| canvas.addLayer | Create layer from text, image, shape, generated asset, or template element. |
| canvas.transformLayer | Move/resize/rotate layer with snapping, guides, and aspect lock. |
| canvas.editText | Update text content and typography with font fallback. |
| canvas.applyMask | Attach editable mask to layer and preserve original media. |
| canvas.removeBackground | Call provider/local service, create masked layer, keep original asset. |
| canvas.generativeFill | Use selection/mask plus prompt to create real generated result and store provenance. |
| canvas.smartResize | Create new page/artboard variant and reposition elements by constraints. |
| canvas.export | Create render/export job for selected pages and formats. |

## 9.5 Image editor AI tools

| AI tool | Input | Output |
|---|---|---|
| Prompt-to-design | Brief, platform, brand kit. | Editable page with headline, visuals, layout, CTA, layers. |
| Brand Match | Existing design plus brand kit. | New version using selected fonts, colors, tone, and visual rules. |
| Product Scene Generator | Product image, audience, scene prompt. | Realistic product scene with shadows/reflections and editable layers. |
| Background Studio | Selection/product/subject and prompt. | Generated, replaced, or extended background. |
| Thumbnail Optimizer | Video topic/script and target platform. | Thumbnail variants with hook text and high-contrast layout. |
| Carousel Builder | Article/script/topic. | 5-10 page carousel with consistent design and editable copy. |
| Batch Variants | Base design and variant instructions. | Multiple ad/post variants with stored experiment tags. |
| Rights Checker | All page assets. | Rights/provenance checklist and missing metadata warnings. |

## 9.6 Image editor acceptance requirements

- A user can create a social image from a brief, generate real image assets, edit layers and text, resize to multiple platforms, export PNG/JPG/WebP, and publish or schedule.
- A user can create a multi-page carousel, reorder pages, edit each page, export pages, and preserve brand styling.
- All generated edits store prompt, provider, model, input asset, output asset, cost, rights metadata, and safety status.
- Canvas state survives refresh, logout/login, and browser crash through autosave/versioning.

# 10. AI Gateway and Real Provider Integration

The app must not depend directly on one model in the UI. All AI calls go through the AI Gateway. The gateway selects, validates, bills, moderates, retries, and records provider jobs. The app must use real provider adapters. Missing credentials produce setup-required states.

## 10.1 Required provider categories

| Category | Examples of compatible providers | Required app capability |
|---|---|---|
| LLM | OpenAI, Anthropic, Google Gemini, others. | Brief analysis, scripts, storyboards, captions, metadata, structured edit operations. |
| Image generation/editing | OpenAI image models, Google Gemini image, Adobe Firefly, Stability, Runway image, others. | Text-to-image, image-to-image, inpainting, outpainting, product scenes, style refs. |
| Video generation/editing | OpenAI video models, Google Veo, Runway, Luma, Pika, Kling where available. | Text-to-video, image-to-video, scene extension, reference images, variations. |
| Voice/TTS | ElevenLabs, OpenAI audio, Google, Azure, others. | Voiceover, voice library, pacing, style, multilingual support. |
| Transcription/captions | OpenAI speech-to-text, Deepgram, AssemblyAI, Google, local Whisper where deployed. | Word-level timestamps and caption blocks. |
| Lip-sync/avatar | HeyGen, Synthesia, D-ID, Tavus, others where licensed. | Presenter clips, lip-sync, multilingual dub with consent controls. |
| Music/SFX | ElevenLabs SFX, Soundraw, Mubert, stock/music APIs where licensed. | Background music, sound effects, rights metadata. |
| Moderation/safety | OpenAI moderation, provider-native moderation, custom policy rules. | Pre/post generation safety checks across text/image/audio/video metadata. |
| Translation/dubbing | LLM translation plus TTS/dubbing providers. | Translate scripts, captions, metadata, audio timing. |

## 10.2 Provider adapter interface

```
interface AIProviderAdapter<TInput, TOutput> {
  providerKey: string;
  taskTypes: AITaskType[];
  validateCredentials(): Promise<ProviderHealth>;
  estimateCost(input: TInput, model: string): Promise<CostEstimate>;
  validateInput(input: TInput): Promise<ValidationResult>;
  submitJob(input: TInput, options: ProviderJobOptions): Promise<ProviderSubmittedJob>;
  pollJob(providerJobId: string): Promise<ProviderJobStatus<TOutput>>;
  cancelJob(providerJobId: string): Promise<void>;
  normalizeOutput(output: unknown): Promise<TOutput>;
  mapError(error: unknown): NormalizedProviderError;
}

type AITaskType =
  | "llm" | "image.generate" | "image.edit" | "video.generate" | "video.edit"
  | "voice.tts" | "voice.clone" | "transcription" | "lipsync" | "music" | "sfx"
  | "moderation" | "translation" | "embedding";

type ProviderHealth = {
  configured: boolean;
  available: boolean;
  quotaRemaining?: number;
  rateLimitResetAt?: string;
  message?: string;
};
```

## 10.3 AI job lifecycle

| Status | Meaning | Required user behavior |
|---|---|---|
| draft | Job created but not submitted. | Show estimated cost and needs approval if expensive. |
| queued | Queued in internal worker system. | Show queue position/progress. |
| running | Provider job submitted or processing. | Show provider, model, elapsed time, cancel option if supported. |
| succeeded | Provider returned output and asset saved. | Show preview, add to asset library, record usage. |
| failed | Job failed after retry/fallback rules. | Show actionable error, do not charge final credits if no usable output. |
| cancelled | User or system cancelled. | Do not use output; charge only if provider already billed and policy says so. |
| blocked | Safety/moderation blocked. | Show policy reason category and safe edit suggestions if allowed. |
| setup_required | Provider credentials/approval missing. | Show setup instructions; do not pretend success. |

### 10.4 Routing rules

- Default users use Auto Best. Power users/admins may choose approved providers when enabled.
- Routing considers quality, reliability, latency, cost, task support, commercial rights, safety, user plan, and enterprise allowed-provider list.
- If a provider fails due to transient error, retry according to provider policy. If still failed, fallback to next compatible provider when available and when user cost settings allow.
- Never charge final credits twice for duplicate or retried internal jobs.
- Store provider, model, version/date, prompt, negative prompt, references, input hashes, output assets, safety status, cost estimate, final cost, latency, and error details.

### 10.5 AI provider configuration UI

- Owners/admins can add provider credentials in /app/providers.
- Each provider shows configured/unconfigured, health, model list, allowed task types, daily quota, rate limits, current errors, and last successful job.
- Credentials are encrypted and never returned to the browser after storage.
- Provider order, cost caps, model allowlist, and workspace-level restrictions are configurable.

## 11. Prompt System and Agentic Workflows

Prompts must be versioned and structured. The app should not rely on random one-off prompts hidden in code. Every prompt template must have inputs, schema output, safety constraints, model defaults, and version history.

### 11.1 Required prompt templates

| Template | Purpose | Output schema |
|---|---|---|
| brief_analyzer | Analyze user idea, audience, platform, brand, risks. | goal, audience, tone, platform, format, risk_flags, missing_info. |
| hook_generator | Generate hook options. | hooks[] with text, rationale, style, platform_fit. |
| script_writer | Write video narration/on-screen text/CTA. | script_sections, narration, onscreen_text, cta, duration_estimate. |
| storyboard_generator | Break script into scenes. | scenes[] with duration, visual_prompt, camera, audio, assets. |
| image_layout_planner | Create post/ad/thumbnail/carousel layout. | headline, hierarchy, layers, colors, typography, prompts. |
| image_prompt_enhancer | Convert brief to provider-ready image prompt. | positive_prompt, negative_prompt, aspect_ratio, references. |
| video_prompt_enhancer | Convert scene to provider-ready video prompt. | prompt, camera, motion, duration, references, safety_notes. |
| caption_hashtag_generator | Create social metadata. | title, description, hashtags, alt_text, cta. |
| editor_assistant | Convert user edit instruction to structured operations. | operations[] with target ids and reversible changes. |
| analytics_recommender | Suggest improvements from performance. | recommendations[] with reason, expected impact, action. |

### 11.2 Structured edit operation rule

```
type EditorAssistantResult = {
  summary: string;
  requiresUserApproval: boolean;
  operations: Array<
    | { type: "timeline.trimClip"; clipId: string; startMs?: number; durationMs?: number }
    | { type: "timeline.addCaption"; trackId: string; startMs: number; durationMs: number; text: string; styleId: string }
    | { type: "timeline.applyEffect"; clipId: string; effect: EffectInstance }
    | { type: "canvas.updateText"; layerId: string; text: string }
    | { type: "canvas.transformLayer"; layerId: string; transform: Partial<LayerTransform> }
    | { type: "generation.requestAsset"; taskType: AITaskType; prompt: string; references: string[] }
  >;
  warnings: string[];
};
```

### 11.3 Agentic workflow requirements

- AI agents can propose edits but destructive changes require preview/approval unless user has enabled auto-apply for that action.
- Every AI-generated plan must be editable before expensive generation starts.
- Every prompt output must be validated against a schema. Invalid output triggers repair or user-visible failure, not silent corruption.
- Prompt versions must be stored so historical project outputs are reproducible as far as provider behavior allows.

# 12. System Architecture and Repository Structure

## 12.1 Chosen stack

| Layer | Required stack |
|---|---|
| Monorepo | Turborepo or Nx with TypeScript throughout. |
| Web app | Next.js App Router, React, TypeScript, Tailwind, shadcn/ui, React Query, Zustand or Jotai, Zod. |
| API | NestJS TypeScript REST API with OpenAPI spec; GraphQL optional later, not required. |
| Database | PostgreSQL with Prisma migrations and typed client. |
| Queue/cache | Redis plus BullMQ workers for AI, media, render, publish, analytics, email. |
| Storage | S3-compatible object storage such as AWS S3, Cloudflare R2, or GCS with CDN and signed URLs. |
| Rendering | FFmpeg worker pipeline; optional GPU workers for advanced effects; browser preview uses proxies and Web APIs where supported. |
| Auth | Clerk/Auth0/Supabase Auth or custom NextAuth-compatible auth; choose one and wire RBAC. |
| Billing | Stripe subscriptions, usage credits, invoices, payment methods, webhooks. |
| Emails | Transactional email provider for invites, verification, alerts, and receipts. |
| Observability | Sentry, OpenTelemetry traces, structured logs, metrics dashboard, uptime checks. |
| Deployment | Frontend on Vercel or equivalent; API/workers on AWS ECS/Fargate/Fly/Render/Kubernetes; managed Postgres and Redis. |

## 12.2 Repository structure

```
/apps
  /web              # Next.js user-facing app and admin UI
  /api              # NestJS API service
  /worker-ai        # AI Gateway job workers
  /worker-media     # metadata, proxy, thumbnail, waveform workers
  /worker-render    # FFmpeg/image render workers
  /worker-publish   # social publishing and polling workers
  /worker-analytics # metrics ingestion and recommendations
/packages
  /ui               # shared UI components
  /config           # eslint, tsconfig, env schemas
  /schemas          # Zod DTOs and OpenAPI shared types
  /db               # Prisma schema, migrations, seed for required system data only
  /auth             # RBAC helpers and permission checks
  /ai-gateway       # provider interfaces, routing, model registry
  /prompts          # versioned prompt templates
  /editor-video     # timeline state, commands, preview helpers
  /editor-image     # canvas state, commands, render helpers
  /media            # asset metadata, transcoding utilities
  /publishing       # platform adapters and preflight rules
  /billing          # Stripe and usage ledger helpers
  /safety           # moderation, consent, rights, policy engine
  /shared           # types, utilities, constants
/infra
  /docker           # local dev services
  /terraform        # production cloud resources
  /github-actions   # CI/CD workflows
/docs
  /api              # generated OpenAPI and examples
  /runbooks         # incident, provider outage, failed publishing, billing
  /architecture     # diagrams and decisions
  /provider-setup   # setup guides for every real provider
  /qa               # test plans and release checklist
```

## 12.3 Service boundaries

| Service | Responsibilities |
|---|---|
| Auth/RBAC Service | Sessions, organizations, memberships, roles, permissions, audit context. |
| Project Service | Projects, folders, statuses, editor docs, versions, comments, review links. |
| Asset Service | Uploads, generated assets, thumbnails, tags, rights, signed URLs, lifecycle. |
| AI Gateway Service | Provider registry, routing, jobs, safety checks, usage ledger, retries, outputs. |
| Media Service | Metadata extraction, proxy generation, thumbnails, waveforms, transcodes. |

| | |
|---|---|
| Render Service | Video/image exports, variants, render progress, final output storage. |
| Publishing Service | OAuth, social accounts, preflight, scheduled jobs, direct posting, polling, manual fallback. |
| Billing Service | Stripe, plans, seats, credits, spending caps, invoices, webhooks. |
| Analytics Service | Post metrics, editor/product usage, normalized dashboards, recommendations. |
| Safety Service | Moderation, consent, rights, watermarks/disclosures, reporting/takedown. |
| Admin/Ops Service | Support tools, provider health, job search, cost dashboard, incidents. |

# 13. Database Schema

Use PostgreSQL and Prisma. The exact Prisma syntax can be implemented by the agent, but every entity below must exist with equivalent fields, indexes, and relations.

| Table | Required important fields |
|---|---|
| users | id, email, name, avatar_url, email_verified_at, status, created_at, updated_at |
| organizations | id, name, slug, plan_id, billing_customer_id, default_workspace_id, status, created_at, updated_at |
| memberships | id, org_id, user_id, role, permissions_json, status, invited_by, joined_at |
| workspaces | id, org_id, name, slug, default_brand_kit_id, settings_json, created_at, updated_at |
| brand_kits | id, org_id, workspace_id, name, logos_json, colors_json, fonts_json, tone, audience_json, forbidden_terms, style_refs_json, product_refs_json, character_refs_json |
| projects | id, org_id, workspace_id, brand_kit_id, type, title, slug, status, platform_targets_json, campaign_id, created_by, updated_at |
| project_versions | id, project_id, version, snapshot_json, created_by, created_at |
| assets | id, org_id, workspace_id, project_id, type, storage_key, public_url_optional, thumbnail_key, proxy_key, mime_type, size_bytes, duration_ms, width, height, rights_json, provenance_json, tags, deleted_at |
| prompts | id, org_id, project_id, template_key, template_version, input_json, output_json, provider_job_id, created_by, created_at |
| provider_credentials | id, org_id, provider_key, encrypted_secret_ref, status, scopes_json, settings_json, created_by, updated_at |
| model_registry | id, provider_key, model_key, task_type, enabled, quality_score, cost_config_json, capabilities_json, updated_at |
| provider_jobs | id, org_id, project_id, task_type, provider_key, model_key, input_json, output_json, status, error_json, cost_estimate_usd, provider_cost_usd, credits_estimate, credits_charged, safety_status, idempotency_key, created_at, completed_at |
| timelines | id, project_id, version, json_doc, duration_ms, width, height, fps, autosave_at, updated_by |
| canvases | id, project_id, version, json_doc, page_count, autosave_at, updated_by |
| render_jobs | id, org_id, project_id, source_type, source_id, format, settings_json, status, progress, output_asset_id, error_json, idempotency_key, created_at, completed_at |
| social_accounts | id, org_id, workspace_id, platform, account_name, platform_account_id, encrypted_tokens_ref, scopes_json, expires_at, status, connected_by |
| publishing_jobs | id, org_id, workspace_id, project_id, asset_id, platform, social_account_id, metadata_json, scheduled_at, status, platform_post_id, idempotency_key, error_json, created_by |
| analytics_metrics | id, org_id, workspace_id, publishing_job_id, platform, metric_key, metric_value, metric_date, raw_json |
| usage_ledger | id, org_id, workspace_id, user_id, provider_job_id, render_job_id, action_type, credits_delta, provider_cost_usd, description, created_at |
| billing_subscriptions | id, org_id, stripe_subscription_id, plan_key, status, current_period_start, current_period_end, seats, included_credits |
| consents | id, org_id, person_name, subject_type, consent_doc_asset_id, scope_json, expires_at, verified_by, created_at |
| comments | id, project_id, author_id, target_type, target_json, body, status, created_at |
| audit_logs | id, org_id, actor_user_id, action, entity_type, entity_id, ip, user_agent, metadata_json, created_at |
| feature_flags | id, key, description, enabled, rules_json, updated_at |

## 13.1 Required indexes and constraints

- Unique organization slug, workspace slug per organization, project slug per workspace.
- Indexes on org_id/workspace_id for all tenant-owned tables.
- Indexes on provider_jobs.status, render_jobs.status, publishing_jobs.status and scheduled_at.
- Unique idempotency_key per org for provider, render, and publishing jobs.
- Foreign keys for all ownership relations; soft delete for assets/projects where needed.
- Audit log append-only behavior. No normal user flow should edit old audit records.

# 14. Backend APIs and DTOs

All APIs must enforce auth/RBAC, validate request bodies with Zod/class-validator, return typed errors, and emit audit logs for sensitive actions.

| Endpoint | Purpose | Input | Output |
|---|---|---|---|
| POST /api/orgs | Create organization | name | organization |
| GET /api/me | Current user/org/workspaces/permissions | - | session context |
| POST /api/projects | Create project | workspaceId,type,title,brandKitId | project |
| GET /api/projects | List projects | workspaceId,filters | paginated projects |
| GET /api/projects/:id | Read project | - | project detail |
| PATCH /api/projects/:id | Update project metadata | title,status,tags | project |
| POST /api/briefs/analyze | Analyze brief with LLM | projectId,brief | provider job or result |
| POST /api/ai/scripts/generate | Generate scripts/hooks | projectId,inputs | provider job |
| POST /api/ai/storyboards/generate | Generate storyboard | projectId,script | provider job |
| POST /api/ai/images/generate | Generate images | projectId,prompt,refs,settings | provider job |
| POST /api/ai/images/edit | Image edit/inpaint/outpaint | assetId,mask,prompt,settings | provider job |
| POST /api/ai/videos/generate | Generate video clip | projectId,prompt,refs,settings | provider job |
| POST /api/ai/voiceovers/generate | Generate voiceover | projectId,text,voice,settings | provider job |
| POST /api/ai/transcriptions | Transcribe audio/video | assetId,language | provider job |
| POST /api/ai/lipsync/generate | Generate lip-sync/avatar | projectId,assetId,audioId,settings,consentId | provider job |
| GET /api/jobs/:id | Read job status | - | job status |
| POST /api/assets/upload-url | Create signed upload URL | filename,mime,size | upload URL and asset draft |
| POST /api/assets/complete-upload | Finalize upload | assetId,storageKey | asset |
| GET /api/assets | Search/list assets | workspaceId,filters | paginated assets |
| PUT /api/timelines/:id | Save timeline | jsonDoc,version | timeline |
| POST /api/renders | Create render/export job | projectId,sourceId,settings | render job |
| POST /api/social/:platform/connect | Start OAuth | workspaceId | redirect URL |
| GET /api/social/:platform/callback | OAuth callback | code,state | social account |
| POST /api/publishing/preflight | Validate publishability | assetId,platform,metadata | warnings/errors |
| POST /api/publishing/jobs | Schedule/publish | assetId,platform,metadata,scheduledAt | publishing job |
| GET /api/publishing/calendar | Calendar jobs | workspaceId,dateRange | jobs |
| GET /api/analytics/workspace/:id | Workspace analytics | dateRange,platform | metrics/recommendations |
| GET /api/billing/usage | Usage ledger | orgId,dateRange | usage |
| POST /api/billing/checkout | Stripe checkout | planKey,seats | checkout URL |
| POST /api/webhooks/stripe | Stripe webhook | raw body | updated billing |

## 14.1 Error response schema

```
type ApiError = {
  error: {
    code: string;
    message: string;
    details?: Record<string, unknown>;
    requestId: string;
    retryable: boolean;
    setupRequired?: {
      provider?: string;
      missing: string[];
      setupUrl?: string;
    };
  };
};
```

## 14.2 Idempotency rule

- Generation, render, billing, and publishing write actions must accept or generate idempotency keys.
- Retries must return the existing job/result when the same idempotency key is used.
- Publishing jobs must never create duplicate social posts on worker retry or API timeout.

# 15. Frontend Architecture

| Frontend area | Required implementation |
|---|---|
| Design system | Shared components for buttons, dialogs, forms, cards, tables, editors, timelines, canvases, toasts, empty/error states. |
| State management | Server state with React Query. Editor local state with Zustand/Jotai and command history. |
| Validation | Shared Zod schemas for forms and API DTOs. |
| Editor performance | Use virtualization, workers, memoized rendering, proxy media, and debounced autosave. |
| Accessibility | Keyboard navigation for common UI, labels, focus states, contrast, captions/alt text support. |
| Internationalization readiness | User content language support now; UI translation architecture ready for later. |
| Error states | Setup required, insufficient credits, provider down, permission denied, quota exceeded, render failed, publish failed, token expired. |

## 15.1 Required UI states per async job

- Not configured: show setup-required with provider/platform name and owner/admin CTA.

- Cost estimate: show credits and possible provider cost before user runs expensive jobs.
- Queued/running: show progress, provider, model, elapsed time, cancel option when supported.
- Succeeded: show preview and actions to add to timeline/canvas, save asset, regenerate, create variant.
- Failed: show normalized error, retry/fallback options, no double charge, support request link.
- Blocked: show safety category and allowed alternatives when appropriate.

# 16. Media, Render, Export, and Storage Pipeline

## 16.1 Storage rules

- All uploaded and generated media must be stored in object storage with tenant-aware keys.
- Assets must have metadata extraction: mime type, size, width, height, duration, codec, fps, bitrate where applicable.
- Create thumbnails for images/videos, proxies for video preview, waveforms for audio, and preview sprites where useful.
- Use signed URLs for private access. Use public temporary URLs only when required by social publishing APIs and expire them when possible.
- Do not store large binary media in PostgreSQL.

## 16.2 Render worker requirements

| Render type | Required behavior |
|---|---|
| Video MP4 | FFmpeg render from timeline JSON, source assets, captions, effects, audio mix, output H.264/AAC MP4. |
| Video variants | Create 9:16, 1:1, 16:9, and custom variants from timeline or duplicates. |
| Thumbnail | Render still frame or image editor output at platform size. |
| Image export | Render canvas pages to PNG/JPG/WebP with optional transparent background. |
| Carousel export | Export individual images plus optional PDF/ZIP package. |
| Proxies | Generate low-res proxy media for editor preview. |
| Waveforms | Generate waveform data for timeline audio tracks. |

## 16.3 Export presets

| Preset | Settings |
|---|---|
| TikTok/Reels/Shorts 9:16 | 1080x1920, H.264, AAC, duration validation, captions safe zones. |
| Square social 1:1 | 1080x1080, H.264/PNG/JPG, safe margins. |
| YouTube thumbnail | 1280x720 JPG/PNG under platform size limit. |
| LinkedIn post image | Common social image dimensions, alt text required in metadata. |
| Website/banner | Custom dimensions, WebP/JPG/PNG. |
| Transparent asset | PNG with alpha for image editor output where supported. |

## 16.4 Render acceptance

- Render jobs must be queued, resumable where practical, and progress-tracked.
- Final output must have correct duration, dimensions, audio sync, aspect ratio, and no unintended black/frozen sections.
- Failed renders must leave source project intact and show clear errors.
- Render outputs become assets with provenance and export settings stored.

# 17. Social Publishing and Scheduling

## 17.1 Platform coverage

| Platform | Required production approach |
|---|---|
| YouTube | OAuth connection, video upload via official API, title/description/tags/privacy, thumbnail where allowed, status tracking. |
| TikTok | Content Posting API where app approval allows direct post or draft upload; validate supported media and creator info. |
| Instagram/Facebook | Meta/Instagram publishing for eligible Business/Creator accounts after app review; media container flow where required; manual fallback when unavailable. |
| LinkedIn | Posts API for personal/company posts where scopes and approval allow; upload media first and reference URNs where required. |
| X/Twitter | Optional based on API pricing/approval; keep adapter interface ready. |
| Pinterest | Add via adapter when business value is prioritized; image/ecommerce workflows. |
| Manual export fallback | Every platform must have a manual export package if direct API access is missing or restricted. |

## 17.2 Publishing job lifecycle

- Draft created with platform, social account, asset, metadata, thumbnail, captions, hashtags, alt text, schedule time, approval state.
- Preflight validates codec, duration, aspect ratio, file size, URL accessibility, account scopes, token expiry, permissions, and platform-specific metadata.
- User approves publish/schedule.
- Scheduler creates idempotent publishing job.
- Worker refreshes token, uploads media/container if needed, sends publish request, stores platform_post_id, and polls/webhooks for status.
- On success, job becomes published and analytics collection begins.
- On failure, job stores normalized error and user sees retry/reschedule/export options.

### 17.3 Publishing metadata

```
type PublishingMetadata = {
  title?: string;
  description?: string;
  caption?: string;
  hashtags: string[];
  altText?: string;
  privacy?: "public" | "unlisted" | "private";
  thumbnailAssetId?: string;
  firstComment?: string;
  platformSpecific: Record<string, unknown>;
};
```

## 18. Billing, Plans, Credits, and Usage Ledger

### 18.1 Plans

| Plan | Required capabilities |
|---|---|
| Free | Limited credits, watermark, limited exports, setup friction allowed but product must work within limits. |
| Creator | No watermark, more credits, HD exports, brand kit, basic publishing. |
| Pro | More credits, premium providers where enabled, advanced editor tools, analytics, templates. |
| Studio | Team seats, multiple brand kits/workspaces, approvals, publishing calendar, higher limits. |
| Agency | Client workspaces, review links, bulk generation, analytics, higher queues, spending controls. |
| Enterprise | SSO/OIDC/SAML where implemented, audit logs, approved providers, custom limits, private storage options, support SLA. |

### 18.2 Usage ledger rules

- Every AI job, render job, publishing paid action, storage overage, and premium feature action must record credits/cost.
- Show estimated credits before expensive actions.
- Failed jobs without usable output should not charge final credits except clearly defined non-refundable provider charges; communicate this in UI.
- Support spending caps per organization and workspace.
- Stripe webhooks must update subscriptions, invoices, payment failures, cancellations, and plan changes.

## 19. Analytics and Recommendation Loop

### 19.1 Required analytics

| Analytics area | Required metrics |
|---|---|
| Activation | Brief started, AI plan generated, first generation, first edit, first export, first publish. |
| Creation | Projects per workspace, assets generated, regeneration rate, selected variant rate, editor session length. |
| Publishing | Scheduled, published, failed, retries, token failures, platform-level success rate. |
| Performance | Views, likes, comments, shares, saves, watch time, click-through where API allows. |
| Cost | Provider cost by category, credits used, gross margin per job, render cost, storage. |
| Quality | Render failures, export failures, provider failure rate, safety blocks, user regeneration reasons. |

### 19.2 Recommendations

- AI should recommend better hooks, posting times, caption styles, thumbnail styles, video lengths, and formats based on workspace analytics.
- Recommendations must show reason and source metrics; do not present unsupported claims as fact.
- Analytics permissions follow workspace and platform account permissions.

# 20. Security, Privacy, Safety, Consent, and Rights

## 20.1 Security requirements

- Encrypt provider credentials and social OAuth tokens at rest.
- Use HTTPS everywhere and secure cookies/session settings.
- RBAC enforced on every API route and worker operation.
- Signed URLs for private assets; no public buckets for user media.
- Input validation and output encoding to prevent injection and XSS.
- Rate limiting for auth, generation, publishing, upload, and admin actions.
- Audit logs for login, credential changes, billing changes, member changes, publish actions, deletion, support access, and provider config changes.
- Data export and deletion request flow for users/orgs, respecting legal/accounting retention requirements.

## 20.2 AI safety requirements

| Risk | Required mitigation |
|---|---|
| Voice cloning misuse | Require explicit consent record before cloning or likeness voice use. Store scope and expiry. |
| Likeness/deepfake misuse | Consent required for recognizable private persons. Block impersonation and harmful/deceptive uses. |
| Copyright/IP risk | Store provenance, user attestations, provider rights metadata, and reporting/takedown workflow. |
| Unsafe content | Moderate prompts and outputs before generation and publishing. Block or review disallowed content. |
| Children/minors | Add stricter likeness and safety review rules. |
| Sensitive user media | Private by default, signed URLs, no training opt-in unless explicitly built and accepted. |
| Watermark/disclosure | Support optional or required AI disclosure/watermark depending on policy, provider, platform, and jurisdiction. |

## 20.3 Rights/provenance metadata

```
type RightsMetadata = {
  source: "uploaded" | "generated" | "stock" | "licensed" | "public-domain" | "unknown";
  ownerUserId?: string;
  providerKey?: string;
  modelKey?: string;
  promptId?: string;
  license?: string;
  commercialUseAllowed?: boolean;
  consentIds: string[];
  aiGenerated: boolean;
  disclosureRequired?: boolean;
  notes?: string;
};
```

# 21. Admin, Support, Observability, and Operations

## 21.1 Admin console

| Admin screen | Required capabilities |
|---|---|
| Users/orgs | Search, view plan/status, workspace count, usage, support notes, audit trail. |
| Provider health | Configured providers, success rate, latency, quota, recent errors, cost. |
| Queue dashboard | AI/render/publish/analytics queues, stuck jobs, retry, cancel, inspect. |
| Cost dashboard | Provider spend, credits charged, gross margin, top orgs, anomalies. |
| Safety dashboard | Blocked jobs, reports, takedown requests, consent records, escalations. |
| Feature flags | Enable/disable features by org/plan/user, rollout percentage, kill switches. |
| Incident banner | Display provider outage or system issue to affected users. |

## 21.2 Observability

- Every request has requestId and traceId.
- Every provider job logs provider/model/job id/status/latency/cost/error without leaking secrets.
- Dashboards for API latency, worker queue depth, render failures, provider failures, publishing failures, Stripe webhook failures, auth errors.
- Alerts for high failure rate, queue backlog, cost spikes, token refresh failures, storage errors, and database issues.

- Runbooks for provider outage, render backlog, social publishing outage, billing webhook failure, data deletion request, and security incident.

# 22. Production QA and Test Requirements

## 22.1 Required test layers

| Test type | Required coverage |
|---|---|
| Unit tests | Schemas, pricing, routing, permissions, editor operations, preflight validation, safety rules. |
| Integration tests | Database, API routes, queues, storage signing, Stripe webhooks, provider adapter credential validation. |
| Editor tests | Timeline operations, canvas operations, autosave, undo/redo, export settings, corrupt document recovery. |
| Render tests | Known timeline/canvas fixtures render to playable/valid outputs with expected dimensions/duration. |
| E2E tests | Signup, onboarding, create video, generate real setup-required/real job depending env, edit, export, publish preflight, billing. |
| Security tests | RBAC bypass attempts, signed URL expiry, token encryption, input validation, rate limits. |
| Performance tests | Editor load, project list, asset library, queue throughput, render throughput, API latency. |
| Accessibility tests | Core pages, forms, keyboard controls, captions/alt text prompts, contrast. |

## 22.2 No fake product behavior QA rule

- User-facing flows must not show generated images/videos/audio unless they came from a real provider job or user upload.
- If a provider is not configured, tests must confirm the app shows setup-required and does not create fake assets.
- Buttons for publish/generate/export must call real services and return real statuses.
- Seed data is allowed only for required system defaults such as plans, templates, and feature flags. Do not seed fake user projects as product truth.

## 22.3 Required acceptance flows

| Flow | Required result |
|---|---|
| New user signup | User creates account, org, workspace, first brand kit, and reaches dashboard. |
| Video build | User creates video project, gets AI plan, generates configured assets or setup-required state, edits timeline, exports MP4. |
| Image build | User creates image project, generates configured assets or setup-required state, edits canvas/layers, exports PNG/JPG. |
| Publishing | User connects a social account where configured, passes preflight, schedules/publishes, and sees status. |
| Billing | User upgrades plan, receives credits/seats, usage ledger updates after job. |
| Safety | Unsafe prompt is blocked or escalated; consent-required voice/likeness flow cannot proceed without consent record. |
| Permissions | Viewer cannot edit or publish; editor cannot change billing; admin can manage workspace. |
| Recovery | Failed AI/render/publish jobs show error and retry/fallback/manual export options. |

# 23. Deployment and DevOps

## 23.1 Environments

| Environment | Purpose | Rules |
|---|---|---|
| local | Developer machine. | Runs app, API, workers, Postgres, Redis via Docker; uses developer-provided provider keys or setup-required states. |
| staging | Production-like QA. | Uses separate database/storage/secrets/social apps/Stripe test mode; runs full E2E and render tests. |
| production | Live customers. | Locked secrets, monitoring, backups, CDN, real billing, real providers, rollback process. |

## 23.2 CI/CD

- On every pull request: lint, typecheck, unit tests, API schema check, Prisma migration check, security scan.
- On main branch: build Docker images, run integration tests, deploy to staging, run E2E smoke tests.
- Production deploy requires passing staging smoke tests and migration safety checks.
- Database migrations must be backward-compatible where possible and include rollback notes.
- Workers and API deploys must support graceful shutdown to avoid losing jobs.

## 23.3 Backups and retention

- Daily automated Postgres backups with tested restore process.
- Object storage lifecycle policies for deleted assets and temporary publishing URLs.
- Audit logs retained according to plan/legal requirements.
- Soft delete projects/assets with restore window; permanent deletion follows explicit retention policy.

# 24. Complete Build Backlog for the AI Agent

The agent should implement in this order to reduce rework. This is implementation order, not reduced scope.

## Foundation

- Create monorepo, TypeScript config, lint, formatting, package scripts, Docker local services.
- Create Next.js web app, NestJS API, shared schemas, Prisma/Postgres, Redis/BullMQ.
- Implement auth, organizations, workspaces, memberships, roles, permissions.
- Implement design system shell, app layout, dashboard, routing, empty/error/loading states.

## Core data and assets

- Implement projects, asset library, uploads, signed URLs, metadata extraction, thumbnails.
- Implement brand kits, templates, project versions, audit logs.
- Implement usage ledger and billing data model.

## AI Gateway

- Implement provider credential storage, health checks, model registry, adapter interface, routing.
- Implement real LLM adapter, real image adapter, real video adapter, real TTS adapter, real transcription adapter, moderation adapter according to available credentials.
- Implement setup-required states for missing credentials and provider approval gaps.
- Implement prompt templates, schema validation, safety checks, cost estimates, job queue, outputs saved as assets.

## Creation flows

- Build create wizard, brief page, AI plan page, generation studio, variants, selected outputs.
- Build campaign package generation and linked child projects.

## Video editor

- Implement timeline JSON state, tracks, clips, operations, autosave, undo/redo.
- Implement preview, asset insertion, trim/split/move/delete, captions, audio controls, transitions, basic effects.
- Implement AI rough cut, captions, auto reframe, B-roll, variants as structured operations.
- Implement render worker and export screen.

## Image editor

- Implement canvas JSON state, pages, layers, operations, autosave, undo/redo.
- Implement text, images, shapes, masks, background remove, generative fill/edit, smart resize, templates.

- Implement image export worker and variant export.

### Publishing

- Implement social account OAuth framework, token vault, platform adapters, preflight validation.
- Implement calendar, scheduled jobs, direct posting where credentials/approval allow, status polling, manual export fallback.

### Billing and analytics

- Implement Stripe checkout/customer portal/webhooks, plans, seats, credits, spending caps.
- Implement analytics ingestion, dashboards, recommendations, cost dashboards.

### Safety, admin, production

- Implement consent records, provenance, moderation, reports/takedowns, watermark/disclosure settings.
- Implement admin console, provider/queue/cost/safety dashboards, feature flags, runbooks.
- Implement CI/CD, staging, production deploy, tests, monitoring, backups, documentation.

## 25. Production Completion Requirements

The agent may call the project complete only when every requirement below is true. This section is not permission to end early; it defines what 100 percent built means for this specification.

| Area | 100 percent built requirement |
|---|---|
| Core SaaS | Auth, orgs, workspaces, RBAC, projects, assets, brand kits, templates, billing, admin, audit logs all work in production environment. |
| AI | Real provider adapters exist for configured categories; setup-required states exist for missing credentials; jobs track cost/status/output/provenance. |
| Video | A user can complete full video flow from brief to generated assets to timeline editing to MP4 export and publish/schedule/manual export. |
| Image | A user can complete full image flow from brief to generated assets to layer editing to variants/export and publish/schedule/manual export. |
| Publishing | OAuth, preflight, schedule, publish/status handling, retry/idempotency, and manual export fallback are implemented. |
| Billing | Stripe billing, credits, usage ledger, invoices, plan limits, spending caps, and failed-job charging rules work. |
| Safety | Moderation, consent, rights metadata, disclosures/watermarks where configured, report/takedown, and audit logs work. |
| Quality | Unit, integration, editor, render, E2E, security, and performance smoke tests pass in CI/staging. |
| Deployment | Production environment has database, Redis, storage, workers, API, web app, secrets, monitoring, backups, and rollback plan. |
| No fake functionality | No core page or button ships as placeholder. No generated media is displayed unless from real provider/upload/render. Missing external access shows setup-required. |

# Appendix A. Environment Variables

| Variable | Purpose |
|---|---|
| DATABASE_URL | Postgres connection string. |
| REDIS_URL | Redis/BullMQ connection string. |
| APP_BASE_URL | Public app URL. |
| API_BASE_URL | Public API URL. |
| S3_BUCKET / S3_REGION / S3_ACCESS_KEY_ID / S3_SECRET_ACCESS_KEY | Object storage. |
| CDN_BASE_URL | CDN URL for public/temporary assets. |
| AUTH_SECRET and auth provider keys | Session/auth provider configuration. |
| STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET | Billing. |
| OPENAI_API_KEY / GOOGLE_API_KEY / RUNWAY_API_KEY / ELEVENLABS_API_KEY / etc. | AI provider credentials according to configured adapters. |
| YOUTUBE_CLIENT_ID / YOUTUBE_CLIENT_SECRET | YouTube OAuth. |
| TIKTOK_CLIENT_KEY / TIKTOK_CLIENT_SECRET | TikTok OAuth/content posting. |
| META_APP_ID / META_APP_SECRET | Instagram/Facebook publishing where approved. |
| LINKEDIN_CLIENT_ID / LINKEDIN_CLIENT_SECRET | LinkedIn OAuth/posting. |
| SENTRY_DSN / OTEL_EXPORTER_OTLP_ENDPOINT | Observability. |
| ENCRYPTION_KEY | Credential/token encryption key from secret manager. |

# Appendix B. Provider Adapter Checklist

- Credential validation and setup-required state.
- Cost estimate before job submission.
- Input validation and provider-specific limits.
- Safety pre-check and post-check where applicable.
- Submit, poll, cancel, normalize output, normalize error.
- Rate-limit handling and retry policy.
- Output asset download/upload to app storage.
- Provenance metadata: provider, model, prompt, refs, date, cost, rights, safety.
- Unit and integration coverage with provider sandbox/real test credentials where available.

# Appendix C. Editor Keyboard Shortcuts

| Shortcut | Required action |
|---|---|
| Space | Play/pause video preview. |
| Ctrl/Cmd+Z | Undo. |
| Ctrl/Cmd+Shift+Z | Redo. |
| S or B | Split/blade selected clip at playhead where editor focus allows. |
| Delete/Backspace | Delete selected clip/layer. |
| Arrow keys | Nudge selected clip/layer or move playhead depending focus. |
| Ctrl/Cmd+C/V | Copy/paste selected clip/layer. |
| Ctrl/Cmd+D | Duplicate selected clip/layer/page. |
| Ctrl/Cmd+S | Force save current editor document. |
| + / - | Zoom timeline/canvas. |

# Appendix D. Required Error States

| Error state | Required user message behavior |
|---|---|
| Provider not configured | Explain which provider/category is missing and show admin setup CTA. |
| Provider quota exceeded | Explain provider limit, retry time if known, and fallback option if available. |
| Insufficient credits | Show credits needed, current balance, upgrade/buy credits CTA. |
| Unsafe request | Show blocked category and safe alternatives where allowed. |
| Render failed | Show reason, retry, duplicate project, contact support, and preserve source edits. |
| Publish token expired | Show reconnect CTA and keep job as draft/failed without duplicate post. |
| Platform validation failed | Show exact platform requirement: duration, size, codec, aspect, caption, thumbnail, scope. |
| Permission denied | Show role limitation and request-access path. |
| Network/server error | Show retry and request ID. |

# Appendix E. Official Documentation Links to Verify During Build

Provider APIs and platform rules change. The agent must verify the current official documentation while implementing adapters. Use this appendix as the starting list, not as permanent truth.

- OpenAI API documentation: video generation, image generation/editing, audio, moderation, structured outputs.
- Google AI/Gemini API documentation: Veo video generation, image generation/editing, Gemini models, safety.
- Runway API documentation: video/image generation, authentication, task polling, model versions.
- ElevenLabs documentation: TTS, voice cloning, speech-to-text, sound effects, consent/safety terms.

- YouTube Data API: OAuth, videos.insert upload, thumbnails, quota, privacy/status fields.
- TikTok Content Posting API: OAuth, direct post/draft, media requirements, creator info, app approval.
- Meta/Instagram Graph API: content publishing, media containers, Business/Creator account constraints, app review.
- LinkedIn Posts API: asset upload, URNs, organic/company posts, scopes, app review.
- Stripe documentation: subscriptions, invoices, credits/usage approach, customer portal, webhooks.
- FFmpeg documentation: transcoding, filters, codecs, H.264/AAC output.
- MDN WebCodecs and WebGPU documentation for browser editor capabilities.

# Final Instruction to the AI Agent

**Build the complete app. Do not build a pitch deck. Do not build a demo shell. Do not fake core functions.**

The required result is a production SaaS application where a real user can create video and image projects, generate assets through configured real AI providers, edit them in real editors, export real files, publish or schedule through real integrations where approved, manage billing and credits, collaborate with teams, view analytics, and operate safely with moderation, consent, provenance, audit logs, and admin controls.